

SIH 2026 Prototype Team Brief & Demo Runbook

Problem Statement: SIH26190 - Secure Digital Document Management System for Legal and Investigation Documents

Organization: Ministry of Home Affairs (NCRB / Women Safety Division as listed in current SIH problem-statement mirrors)

Purpose of this document: Make every team member capable of explaining the problem, solution, security model, demo flow, GitHub submission, prototype limitations, and likely jury questions.

Important implementation note: This brief is grounded in the SIH26190 problem statement and the project details available in the current chat. The exact current source code from the earlier "Review SIH prototype caveats" exchange is not accessible in this chat state. Therefore, any implementation-specific statement below that says "verify in code" must be checked against the final repository before presentation. Do not claim encryption, hashing, versioning, or storage behavior unless the code actually implements it.

1. The Problem

Legal and investigation workflows handle highly sensitive records such as FIRs, police reports, witness statements, charge sheets, evidence records, forensic reports, court filings, legal notices, and judgments. Fragmented paper/digital systems make documents harder to locate, easier to mishandle, difficult to audit, and vulnerable to unauthorized access or modification.

The SIH statement asks for a secure, centralized, scalable and intelligent document management platform that preserves confidentiality, integrity, accessibility, legal validity and evidentiary integrity.

- Centralize and digitize legal/investigation documents.
- Restrict access to authorized users.
- Prevent or detect unauthorized modifications.
- Maintain a complete audit trail of document activity.
- Provide efficient search and retrieval.
- Enable controlled collaboration between authorized stakeholders.
- Support legal/regulatory compliance and evidentiary integrity.

2. Proposed Solution - What the Prototype Should Demonstrate

The prototype should feel like a secure case-document workspace rather than a generic file drive. A user signs in, works within cases, uploads documents, and receives only the actions allowed by their role. Every sensitive action should be attributable to a user and time.

- **Case-based document organization:** documents belong to a case/matter, not an unstructured public folder.
- **Role-based access control (RBAC):** roles determine who can view, upload, edit/replace, approve, download, share, or administer.
- **Integrity protection:** document hashes and/or a tamper-evident audit chain can demonstrate that content has not silently changed.
- **Auditability:** record login, upload, view, download, update, permission change, and other sensitive events.
- **Search and retrieval:** search by case/document metadata and, where implemented, indexed text.
- **Version control:** preserve earlier versions instead of silently overwriting legal evidence.
- **Controlled sharing:** access is granted deliberately and can be revoked according to role/policy.

3. Access Levels and Control

Role	Typical permissions	What to emphasize in demo
Administrator	User/role management, policy configuration, case-level oversight; should not be able to edit evidence	Administrative role is separated from evidence editing
Investigating / Case Officer	Create or manage assigned cases, upload records, view permitted documents	Operations, add support only the scope they need.
Supervisor / Approver	Review records, approve/reject actions, access broader case scope, higher authority	Higher authority adds oversight, not invisible alteration.
Legal / Court / Authorized Reviewer	View/download only the records explicitly shared with that role or case	Least-privilege access.
Auditor / Compliance	Read audit history and integrity information; ideally cannot alter source	Separation of duties and accountability.

Use the role names that actually exist in your prototype. Do not invent capabilities during the presentation. If your build has fewer roles, explain that the architecture supports finer-grained RBAC as a production extension.

4. Security Model - What to Say and What Not to Overclaim

A strong explanation separates **authentication**, **authorization**, **confidentiality**, **integrity**, and **auditability**. Judges often ask these separately.

- **Authentication:** verifies who the user is.
- **Authorization / RBAC:** determines which cases, documents and actions that user may access.
- **Confidentiality:** sensitive document bytes and metadata should be protected from unauthorized users; in production this normally includes TLS in transit and encryption at rest.
- **Integrity:** a cryptographic hash can reveal content changes. A hash-chained or otherwise append-only audit design makes history tampering easier to detect.
- **Auditability:** sensitive activity is logged with actor, action, target and timestamp.
- **Versioning:** a changed legal document should create a new version rather than destroying the prior state.
- **Least privilege:** users receive only the minimum access required for their role/case.

Security claim rule: if the current prototype merely stores files in a local folder behind login, say exactly that: “The prototype demonstrates controlled application-layer access; production deployment would use encrypted object storage, TLS, hardened secret management and institutional identity integration.” Do not call a local folder ‘encrypted secure storage’ unless it is actually encrypted.

5. Where Uploaded Files Are Stored - Localhost Prototype

Because the current demo is described as a localhost prototype, uploaded files normally remain on the machine running the backend unless you have explicitly connected cloud/object storage. In many web prototypes the file bytes are written to an uploads/storage directory while the database stores metadata such as file name, case ID, owner, path, permissions, version and hash.

Verify before demo: upload one harmless test PDF, then inspect your project directory/database and confirm exactly where the file and its metadata appear. Your team should be able to answer: “Where are the bytes?”, “Where is the metadata?”, and “What prevents another role from fetching them?”

Is there a file-count limit? Local application code often has no explicit count limit unless you implemented one. The practical limit is disk capacity plus any per-file upload-size limit in the backend/web framework. Therefore, do not quote a maximum number of files unless the code defines one. State the current per-file size limit if one exists and explain that production scale would move file bytes to scalable object storage.

6. Recommended Production Storage Architecture

- Encrypted object/document storage for file bytes.
- Relational database for users, roles, cases, document metadata, versions and permissions.
- Cryptographic hash stored alongside each immutable version.
- Append-only or hash-chained audit records.
- TLS/HTTPS for all network traffic.
- Secrets/keys outside source code and outside GitHub.
- Backups, retention policies and disaster recovery appropriate to legal records.
- Malware scanning and file-type validation for uploads.

7. The Demo Story to Rehearse

Rehearse one clean story end-to-end. A short, reliable flow is stronger than clicking every page.

1. **30-second problem:** “Sensitive legal/investigation documents are fragmented, hard to retrieve, vulnerable to unauthorized access/modification, and difficult to audit.”
2. **10-second solution:** “Our system centralizes case documents and adds role-based access, integrity protection, searchable retrieval, version history and a complete audit trail.”
3. **Login as operational user:** open an existing demo case.
4. **Upload a harmless sample document:** show case metadata and the new document entry.
5. **Show security metadata:** hash/version/owner/classification only if actually implemented.
6. **Search:** retrieve the document by case/document keyword or metadata.
7. **Show access control:** sign in as a restricted user and prove a forbidden action is unavailable or rejected server-side.
8. **Show controlled update:** if versioning exists, upload/revise the document and show both versions.
9. **Show audit trail:** point to who uploaded/viewed/updated what and when.
10. **Close with impact:** faster retrieval, reduced tampering risk, clearer accountability, controlled collaboration and a path to institution-scale deployment.

8. Rehearsal Schedule

- **Run 1 - Functional:** click through slowly and note every bug, broken link, slow action or confusing label.
- **Run 2 - Narrated:** one presenter speaks while another operates the laptop.
- **Run 3 - Role handoff:** every team member explains one module in under 45 seconds.
- **Run 4 - Jury interruption:** teammates interrupt with questions about security, storage, scale, novelty, compliance and failure cases.
- **Run 5 - Offline/no-internet:** demonstrate entirely on localhost with Wi-Fi disabled if possible.
- **Run 6 - Cold start:** restart laptop/server and launch from scratch using only the README/demo checklist.
- **Final:** record a screen demo as backup and keep sample files, credentials and commands in one demo folder.

9. Localhost vs Deployed App

Localhost is acceptable for an internal live demonstration if your institute has not required a public deployment. In fact, localhost can be more reliable because it avoids venue Wi-Fi. The important limitation is that a localhost URL such as 127.0.0.1 or localhost works only on the machine running the app; judges cannot open that link from their own devices.

For the live presentation, you can simply run the backend/frontend on your laptop and open the local web address in the browser. You do **not** need to “download the app” if it is a web application. A desktop/mobile installer is only needed if your prototype was intentionally built as an installable application.

Best demo posture: localhost live demo + GitHub repository + README setup instructions + short backup screen recording. A hosted URL is a bonus, not a substitute for a working local build, unless your institute explicitly requires deployment.

10. Demo-Day Startup Checklist

1. Charge laptop fully; carry charger/adaptor.
2. Keep the final project in a clean, known folder.
3. Close unrelated apps and notifications.
4. Start database/service dependencies first.
5. Start backend server.
6. Start frontend server if separate.
7. Open the demo URL in a clean browser tab.
8. Test login for every demo role.
9. Keep 2-3 harmless sample PDFs ready.
10. Preload one case with realistic dummy data so the UI is not empty.
11. Verify audit trail and role restrictions before presenting.
12. Keep terminal windows organized so restart commands are obvious.
13. Keep a screen-recorded demo available locally in case the live build fails.

11. GitHub - Updating an Existing v0 Repository to the Current Prototype

Do not make a brand-new unrelated repository unless your registration rules require it. Preserve the existing v0 repository history and publish the updated prototype cleanly.

Recommended workflow:

- 1) Back up the current working prototype folder.
- 2) Open Terminal and clone/pull the existing v0 repository into a clean folder.
- 3) Create a branch: `git switch -c prototype-v2`
- 4) Replace/update the v0 application files with the current prototype files.
- 5) Add/update `.gitignore` BEFORE staging. Exclude `.env`, secrets, private keys, database dumps containing sensitive data, uploaded demo evidence, `node_modules`, virtual environments, build caches and local storage folders.
- 6) Add a README containing: problem statement, solution, features, architecture, tech stack, security design, roles, setup, demo credentials using dummy accounts, limitations and screenshots.
- 7) Run the app from the clean repository and make sure another teammate can follow the README.
- 8) `git status`
- 9) `git add .`
- 10) `git commit -m "Update prototype for SIH internal hackathon"`
- 11) `git push -u origin prototype-v2`
- 12) On GitHub, open a pull request into main, review the diff, and merge once the demo build is verified.
- 13) Pull main locally and test again.
- 14) Optional but useful: create a tag/release such as `internal-hackathon-v2` so the judged snapshot cannot be confused with later edits.

What link to submit: if the form literally says "GitHub Demo link", submit the repository URL (preferably the main branch containing the final prototype). Put the run instructions and, if available, hosted demo URL/video near the top of README. A localhost URL should never be submitted as an external demo link because it only works on your machine.

12. README Minimum Contents

- Project title + SIH26190 problem statement.
- 1-paragraph problem and solution.
- Feature list.
- Architecture diagram.
- Tech stack.
- Security model and RBAC matrix.
- How document bytes and metadata are stored in the prototype.
- Installation/setup commands.
- How to launch frontend/backend.
- Demo accounts with dummy credentials only.
- Demo flow.
- Known limitations and production roadmap.
- Screenshots or a short GIF/video link if permitted.

13. Tech Stack - What to Tick

The exact screenshot/options are not available in the current chat, so only tick technologies that are actually in the repository. For a typical web DMS, the relevant categories are likely:

- **Web Development / Full Stack** - if the prototype is a browser-based frontend + backend.
- **Cybersecurity** - definitely relevant to SIH26190.
- **Database** - if you use SQLite, PostgreSQL, MySQL, MongoDB, etc.
- **Cloud** - only if your prototype actually uses cloud services/storage/deployment.
- **Blockchain** - only if blockchain is genuinely implemented, not merely planned.
- **Artificial Intelligence / Machine Learning** - only if the prototype actually performs AI-based classification, OCR, semantic search, summarization, etc.
- **React / JavaScript / TypeScript / Node.js / Python / Flask / FastAPI / Django** - tick only the exact frameworks/languages present in package files/source.

Do not tick technologies just because they appear in the SIH description. Judges can immediately ask where they are used.

14. Submission Readiness Gate

Area	Ready when...
Core workflow	Login -> case -> upload -> retrieve works reliably.
RBAC	At least two roles visibly receive different permissions, enforced by backend.
Integrity	Hash/tamper or version mechanism is demonstrable, or clearly labelled roadmap if not implemented.
Audit	Sensitive actions produce readable audit entries.
Search	A document can be found without browsing folders manually.
Storage explanation	Team can state exactly where bytes + metadata are stored.
Security honesty	No unsupported claims such as 'encrypted' or 'blockchain'.
Repository	Clean repo, no secrets, current code, working README.
Demo	Cold-start rehearsal succeeds; backup screen recording exists.
Presentation	Every member knows problem, architecture, security, impact and their module.

15. Does This Fit SIH26190?

Yes - conceptually, a prototype centered on secure document upload/storage, role-based control, restricted sharing, integrity protection and auditing is directly aligned with SIH26190. The fit becomes strong only if the prototype demonstrates more than a normal cloud-drive UI.

The official brief specifically asks for secure centralized storage, confidentiality, protection against unauthorized modifications, audit trails, efficient retrieval, authorized collaboration, compliance and evidentiary integrity. Those should therefore be the spine of your demo.

Highest-priority gaps to fix if missing:

- Backend-enforced RBAC (not just hiding buttons).
- Audit trail for sensitive actions.
- Integrity/tamper evidence (hashes are enough for a prototype if explained correctly).
- Version history instead of silent overwrite.
- Useful search/retrieval.
- A clear case/evidence workflow rather than a generic upload/download page.

Novelty warning: a basic document management system is easy for judges to compare with existing enterprise products. Your differentiation should be legal/investigation-specific: evidentiary integrity, least-privilege case access, immutable history, chain-of-custody style auditability, controlled cross-department sharing, and a credible path to institutional deployment.

16. Likely Jury Questions - Team Answers to Rehearse

Q: Why not Google Drive/SharePoint?

Because the target workflow needs case-specific least-privilege access, evidentiary integrity, version history, auditable sensitive actions and legal/investigation-specific controls rather than generic collaboration.

Q: What stops an unauthorized user from downloading a file?

The backend checks identity, role/case permission and requested action before serving the document. UI hiding alone is not treated as security.

Q: How do you know a document was not altered?

Each stored version can be hashed; changing the bytes changes the hash. The history/audit design then records version and actor information.

Q: Where is the file actually stored?

Answer with your exact prototype implementation: local storage directory/object store + metadata database. Do not improvise.

Q: Can an admin secretly edit evidence?

A well-designed system separates administration from evidence modification and records privileged actions in an auditable history.

Q: How does it scale?

Move file bytes to scalable encrypted object storage, use a production relational database/search index, cache carefully, and horizontally scale application services while keeping access control centralized.

Q: What if internet fails?

For the internal demo, localhost remains operational. A production design can support controlled offline workflows only with strict synchronization and conflict/audit rules.

17. Team-Lead Action Plan

Since you and one teammate have done most of the implementation, your biggest leadership task now is not writing more code yourself - it is reducing single-person dependency.

- 1. Freeze features.** Stop adding cosmetic extras unless they fix a judged criterion or a demo blocker.
- 2. Split ownership.** You own overall narrative/demo; the other primary builder owns technical recovery/restart. Assign each remaining member one real area: problem/impact, security/RBAC, architecture/tech stack, testing/demo support.
- 3. Make every member operate the prototype once.** Nobody should see the UI for the first time on demo day.

4. **Give each member 3 questions.** They must know the answer without looking at you.
5. **Create a single source of truth.** Final repo + README + this runbook + sample files + credentials.
6. **Protect the demo branch.** No last-minute unreviewed changes on presentation day.
7. **Run a failure drill.** Backend stops, wrong password, uploaded file missing, no Wi-Fi. Team should know recovery steps.
8. **Present the team as a team.** You do not need to pretend contribution was equal, but distribute speaking roles so the project does not look dependent on two people.

18. Final 60-Second Pitch Structure

Problem -> consequence -> solution -> security differentiator -> live proof -> impact.

Example structure: “Legal and investigation records are sensitive, fragmented and difficult to audit. Our prototype centralizes them by case while enforcing role-based access. Every important action is attributable, document integrity/version history can be verified, and authorized users can retrieve records quickly without exposing them to everyone. In the demo we will upload a case document, prove that another role cannot modify it, retrieve it by search, and show the resulting history/audit record. The same architecture can move from localhost to hardened institutional storage and identity systems for deployment.”

Sources consulted for the problem-statement alignment: SIH26190 listings/mirrors reproducing the current SIH 2026 brief, including the Ministry of Home Affairs statement and SIH problem-statement datasets. Always treat the official SIH portal and your institute's internal-hackathon instructions as authoritative if they differ.